CGI AWS Integration Architect — L2 Technical Deep-Dive (Part 2)

Kubernetes Deep-Dive + Validated K8s Answers + Scenario Questions

> **Role:** AWS Integration Architect | **Round:** L2 Technical (1 Hour) **Focus:** Kubernetes expertise, EKS operations, and L2 scenario-based questions

---

SECTION 3: KUBERNETES DEEP-DIVE (Validated & Enhanced Answers)

> ✅ = Your original answer validated and kept 🔧 = Enhanced with corrections or additions ➕ = New content added

---

Q11. What is a service mesh, and why is it used in Kubernetes?

✅ **Your answer is excellent — validated with minor enhancements:**

A service mesh is an infrastructure layer that manages **east–west traffic** between microservices in Kubernetes by externalizing networking concerns from application code. Instead of each service implementing its own logic for retries, timeouts, encryption, or observability, a service mesh handles these transparently at the platform level.

It is implemented using a **sidecar proxy model**, where lightweight proxies (typically **Envoy**) are injected alongside application pods to intercept all service-to-service communication.

Why service meshes are used:

Concern	What the Mesh Provides
Traffic management	Load balancing, traffic splitting, canary deployments, circuit breaking, retries — all without code changes
Security	Mutual TLS (mTLS) by default — encrypted communication + strong service identity between all workloads
Observability	Automatic metrics, logs, and distributed traces across all services — no per-service instrumentation needed

Key architectural benefit: Separation of concerns — developers focus on business logic; platform teams centrally define networking, security, and reliability policies.

🔧 **Enhancement — When NOT to use a service mesh:**

- Small clusters with < 10 microservices — overhead isn't justified
- Latency-critical applications where the extra proxy hop (typically 1-3ms) matters
- Teams without Kubernetes platform maturity to operate the mesh
- When simpler alternatives work — e.g., Kubernetes NetworkPolicies for basic traffic control, or application-level libraries for retries

🔧 **Enhancement — Comparison for L2 interviews:**

Mesh	Strengths	Trade-offs
Istio	Feature-rich, large community, traffic management	Complex, resource-heavy, steep learning curve
Linkerd	Lightweight, fast, simple operations	Fewer features than Istio
AWS App Mesh	Native AWS integration, managed control plane	AWS-only, less community adoption
Consul Connect	Multi-platform (K8s + VMs), service discovery built-in	Requires Consul infrastructure

---

Q12. How can you secure a Kubernetes cluster?

✅ **Your 4C security model answer is validated — excellent structure. Adding EKS-specific details:**

4C Model: Cloud → Cluster → Container → Code

Cloud Layer (EKS-specific):

- EKS control plane is managed by AWS — API server, etcd, and controller manager are patched automatically
- **Node groups:** Use managed node groups with AMI auto-updates
- **Private cluster:** API server endpoint private, accessible only via VPN/Direct Connect
- **Security groups:** Separate SGs for control plane and worker nodes with minimal rules
- **VPC:** Private subnets for nodes, no direct internet access (use NAT Gateway or VPC endpoints)

Cluster Layer:

- **RBAC:** Namespace-scoped roles, never grant `cluster-admin` to applications

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: payments
  name: payment-developer
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list", "watch", "create", "update"]
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
```

- **aws-auth ConfigMap:** Maps IAM roles to K8s RBAC groups — controls who can access the cluster
- **Audit logging:** EKS sends API server audit logs to CloudWatch — essential for forensics
- **Admission controllers:** OPA Gatekeeper or Kyverno to enforce policies

Container Layer:

- **Pod Security Standards (PSS):**

```
securityContext:
  runAsNonRoot: true
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false
  capabilities:
    drop: ["ALL"]
```

- **ECR image scanning** on push — block images with critical CVEs
- **Image provenance:** Only allow images from your private ECR registry (Gatekeeper policy)

Code Layer:

- **Secrets:** Use AWS Secrets Manager with CSI Secrets Store Driver (not plain K8s Secrets which are base64-encoded, not encrypted at rest by default)
- **Network Policies:** Calico or VPC CNI network policies for pod-to-pod zero-trust

- **Dependency scanning:** Snyk/Trivy in CI pipeline

Q13. What are Kubernetes sidecar containers, and how are they used?

✅ **Your answer is validated — well-structured. Key enhancement on Native Sidecars:**

🔧 **Enhanced Native Sidecars section:**

Before Kubernetes 1.28, sidecars were just regular containers in the pod spec. This caused two problems:

Problem 1 — Startup race condition: Main app starts before the Envoy sidecar proxy is ready → first requests fail or bypass security

Problem 2 — Shutdown ordering: Pod terminates → sidecar logging container dies before main app → last log lines are lost

Native Sidecars (KEP-753, GA in K8s 1.29) solve both:

```
apiVersion: v1
kind: Pod
spec:
  initContainers:
  - name: istio-proxy
    image: envoy:latest
    restartPolicy: Always    # ← This makes it a native sidecar
    # Starts BEFORE main container, runs for the pod's lifetime
  containers:
  - name: payment-app
    image: payment-service:v2
```

Lifecycle guarantee: `initContainers` with `restartPolicy: Always` start first and are guaranteed running before main containers start. On shutdown, they terminate AFTER main containers.

➕ **Common sidecar patterns in EKS for CGI context:**

Sidecar	Purpose	Example
Envoy proxy	Service mesh traffic management	Istio/App Mesh sidecar
Fluent Bit	Log collection and forwarding	Logs → CloudWatch/OpenSearch
AWS X-Ray daemon	Distributed tracing	Traces → X-Ray
CSI Secrets Store	Mount secrets from Secrets Manager	AWS Secrets → pod volumes
CloudWatch agent	Custom metrics	App metrics → CloudWatch

Q14. What are Kubernetes CRDs, and when should you use them?

✅ **Your answer is validated. Adding practical examples relevant to CGI/retail:**

🔧 **Practical examples for L2:**

Example 1 — Custom integration resource:

```
apiVersion: integrations.canarys.com/v1
kind: RetailIntegration
metadata:
  name: salesforce-order-sync
spec:
  source:
    type: salesforce
    object: Order__c
    trigger: platform-event
  destination:
    type: sqs
    queueArn: arn:aws:sqs:us-east-1:123456:order-queue
  transform:
    type: lambda
    functionArn: arn:aws:lambda:...:transform-order
  retry:
    maxAttempts: 3
    backoffMultiplier: 2
  monitoring:
```

```
    alertOnFailure: true
    slackChannel: "#integration-alerts"
```

A custom controller watches for `RetailIntegration` resources and automatically provisions the Lambda, SQS queue, EventBridge rule, and monitoring — all from a single declarative resource.

Example 2 — Real-world CRDs you'll encounter:

- **ArgoCD:** `Application` CRD defines what to deploy and where
- **Cert-Manager:** `Certificate` CRD to auto-provision TLS certificates
- **External Secrets Operator:** `ExternalSecret` CRD to sync AWS Secrets Manager to K8s Secrets
- **KEDA:** `ScaledObject` CRD to auto-scale pods based on SQS queue depth

---

Q15. How do you backup and restore an etcd cluster in Kubernetes?

✅ **Your answer is validated — both self-managed and EKS sections are correct.**

🔧 **Key addition for EKS context (most relevant for CGI):**

In EKS, you DON'T manage etcd. AWS manages the control plane including etcd. Your backup responsibility is at the application level:

Velero backup strategy for EKS:

```
# Install Velero with AWS plugin
velero install \
  --provider aws \
  --plugins velero/velero-plugin-for-aws:v1.7.0 \
  --bucket my-velero-backups \
  --backup-location-config region=us-east-1 \
  --snapshot-location-config region=us-east-1 \
  --use-node-agent

# Schedule daily backups
velero schedule create daily-backup \
  --schedule="0 2 * * *" \
  --ttl 720h \
  --include-namespaces payments,inventory,notifications
```

```
# Restore to a new cluster
velero restore create --from-backup daily-backup-20260512
```

What Velero backs up:

- All Kubernetes API objects (Deployments, Services, ConfigMaps, Secrets, CRDs)
- Persistent Volume snapshots (EBS snapshots via CSI)
- Namespace-scoped or cluster-scoped filtering

Disaster recovery plan for EKS:

1. Velero backs up to S3 (cross-region replicated)
2. Terraform/IaC can recreate the EKS cluster in DR region
3. Velero restores all workloads and PV data into new cluster
4. DNS (Route53) switches traffic to DR cluster

SECTION 4: ADDITIONAL L2 QUESTIONS (High Probability for CGI)

Q16. How do you handle schema evolution in event-driven integrations?

Answer:

Problem: Producers change event schema (add/remove fields) → Consumers break.

Solution — Schema registry + compatibility rules:

Using EventBridge Schema Registry:

- Auto-discovers schemas from events
- Generates code bindings for consumers
- Version tracks all schema changes

Compatibility strategy:

Type	Rule	Example
Backward compatible	New schema can read old data	Add optional field with default
Forward compatible	Old schema can read new data	Consumer ignores unknown fields
Full compatible	Both directions work	Add optional fields only, never remove

Best practices:

- **Never remove or rename fields** — deprecate and add new ones
- **Always add fields as optional** with sensible defaults
- **Version your events:** `com.retail.orders.v1.OrderCreated` vs `v2`
- **Consumer-driven contract testing** — consumers define what they need; producers validate
- **Dead letter queues** catch deserialization failures from incompatible schemas

---

Q17. How do you design observability for a complex integration platform?

Answer:

Three pillars of observability for integrations:

Metrics (CloudWatch):

```
Dashboard: Integration Health
├── API Gateway: Request count, latency P50/P95/P99, 4xx/5xx rates
├── Lambda: Invocations, errors, duration, concurrent executions, throttles
├── SQS: Messages sent/received, ApproximateAgeOfOldestMessage, DLQ depth
├── Step Functions: Executions started/succeeded/failed/timed-out
├── EventBridge: MatchedEvents, FailedInvocations
└── Business: Orders processed/hour, integration success rate, end-to-end
latency
```

Logs (CloudWatch Logs + Insights):

- Structured JSON logging from all Lambdas with correlation ID

- CloudWatch Log Insights queries:

```
filter @message like /ERROR/
| stats count() by bin(30m)
```

Traces (X-Ray):

- End-to-end trace: API Gateway → Lambda → SQS → Lambda → DynamoDB → External API
- Identify bottlenecks: "The Salesforce API call takes 2.3s out of the total 3.1s"
- Service map showing all integration dependencies

Alerting strategy:

Severity	Condition	Action
P1 Critical	DLQ message count > 0, Step Function failures	PagerDuty → on-call immediate
P2 High	API error rate > 1%, Lambda throttles	Slack alert → investigate within 1 hour
P3 Medium	Latency P99 > threshold, queue age growing	Ticket → next business day
P4 Low	Cost anomaly, approaching quota	Weekly review

Q18. Walk me through a production incident in an integration system and how you resolved it.

Answer (STAR format):

Situation: Retail client's order processing system. Monday morning — 500 orders stuck in SQS queue, DLQ filling up, no orders reaching the OMS.

Task: Restore order processing within SLA (30 minutes) and prevent data loss.

Action:

1. **Triage (5 min):** CloudWatch alarm fired on DLQ count. Checked X-Ray traces — Lambda was timing out calling OMS API.
2. **Root cause (10 min):** OMS had deployed a breaking change over the weekend — renamed an API endpoint. Our Lambda was getting 404s, exhausting retries, messages going to DLQ.
3. **Immediate fix (15 min):**
    - Updated Lambda environment variable with new OMS endpoint
    - Deployed via CI/CD pipeline (pre-tested in staging)
    - Verified new Lambda version processing messages successfully
4. **DLQ recovery (20 min):**
    - Used SQS DLQ redrive to move 500 messages back to the main queue
    - Lambda processed all within 10 minutes
    - Verified all 500 orders appeared in OMS
5. **Prevention:**
    - Added contract testing between our Lambda and OMS API
    - Added CloudWatch alarm on Lambda 4xx error rate (not just DLQ)
    - Established change notification process with OMS team

Result: All 500 orders recovered within 45 minutes. Zero data loss. Contract testing caught 2 similar issues in staging over the next 3 months before reaching production.

Q19. How do you handle large file processing integrations in AWS?

Answer:

Scenario: Retail client receives 5GB product catalog CSV files daily from vendor.

Architecture:

```
Vendor SFTP → AWS Transfer Family → S3 (raw zone)
  │
  ▼ S3 Event Notification
  │
EventBridge → Step Functions (orchestration)
  ├── State 1: Lambda (validate file: schema, row count, checksum)
  ├── State 2: Glue ETL (transform: CSV → Parquet, dedup, normalize)
  │     └── Output → S3 (processed zone)
```

```
    ├── State 3: Lambda (load to DynamoDB/Aurora in batches)
    └── State 4: SNS (notify downstream: catalog updated)
```

Key design decisions:

- **S3 multipart upload** for large files (Transfer Family handles this automatically)
- **Glue over Lambda** for transformation — Lambda has 15-min timeout and 10GB /tmp; Glue handles terabytes
- **Parquet format** for processed data — columnar storage, 80% smaller than CSV, faster analytics
- **Batch loading** to DynamoDB — use BatchWriteItem with 25 items per batch, handle throttling with exponential backoff
- **Checksum validation** — verify file integrity before processing

---

Q20. What questions would you ask a client before designing an integration solution?

Answer:

This is a critical L2 question — shows architectural thinking, not just technical skills.

Category	Questions
Data	What data flows between systems? Volume? Frequency? Format (JSON/XML/CSV/flat file)?
Latency	Real-time (< 1s)? Near-real-time (< 5 min)? Batch (daily/hourly)?
Reliability	What happens if the integration fails? Is data loss acceptable? What's the RPO/RTO?
Security	Data sensitivity? PII? PCI? Compliance requirements (HIPAA/SOC2/GDPR)?
Systems	What systems are involved? On-prem or cloud? APIs available or file-only? Authentication methods?
Scale	Current volume? Expected growth? Peak vs. average load?

Category	Questions
Error handling	What should happen when a message fails? Retry? Alert? Manual review?
Ordering	Does message order matter? Is idempotency required?
Existing	Are there existing integrations? What technology (ESB/MQ/custom)? What works, what doesn't?
Teams	Who owns each system? Change management process? SLA expectations?

SECTION 5: QUICK-FIRE QUESTIONS (Likely in Final 15 Minutes)

Q21. SQS Standard vs FIFO — when do you choose each?

Aspect	Standard	FIFO
Throughput	Unlimited	300 TPS (3,000 with batching)
Ordering	Best-effort	Strict FIFO per message group
Delivery	At-least-once (possible duplicates)	Exactly-once (5-min dedup window)
Use case	High-volume async processing	Order-sensitive: payments, inventory updates

My default: Standard SQS (higher throughput, cheaper). Switch to FIFO only when strict ordering or deduplication is a hard requirement.

Q22. Lambda cold start — how do you minimize it?

- **Provisioned concurrency:** Pre-warm N instances (costs money but eliminates cold starts)
- **SnapStart (Java):** Snapshots initialized JVM state — reduces Java cold start from 5s to <1s
- **Smaller deployment packages:** Remove unused dependencies, use Lambda layers for shared code
- **Choose runtime wisely:** Python/Node.js cold starts ~200ms; Java/C# can be 2-5s
- **Keep connections alive:** Initialize DB connections and SDK clients outside the handler
- **ARM64 (Graviton):** 10-20% faster startup and 20% cheaper

---

Q23. How does Terraform state locking work and why is it important?

- **State file** (`terraform.tfstate`) records current infrastructure state
- **Remote backend (S3)** stores state centrally so teams can collaborate
- **DynamoDB locking table** prevents two people running `terraform apply` simultaneously
- **Without locking:** Two concurrent applies can corrupt state or create duplicate resources
- **State encryption:** S3 SSE-KMS encrypts state at rest (contains sensitive values like passwords)

Q24. Explain the difference between blue-green and canary deployments.

Aspect	Blue-Green	Canary
Approach	Two identical environments; switch all traffic at once	Gradually shift traffic (5% → 25% → 100%)
Rollback speed	Instant — switch back to blue	Fast — route 100% back to old version

Aspect	Blue-Green	Canary
Risk	All-or-nothing switch	Controlled exposure, catch issues early
Cost	Higher — two full environments running	Lower — only one extra instance/pod needed
Best for	Database schema changes, big-bang releases	API changes, feature rollouts

In EKS: Argo Rollouts for canary with automatic rollback based on CloudWatch metrics.
In Lambda: Weighted aliases (10% → new version, 90% → current). CodeDeploy manages traffic shift.

Q25. You're in a meeting with a retail client. They say: "We want real-time inventory across all 500 stores." How do you respond?

Answer:

"Let me understand what 'real-time' means for your business. There are three levels:

1. **True real-time (< 1 second):** POS transaction → immediate stock update visible everywhere. Achievable but expensive — requires event streaming (Kinesis/EventBridge), DynamoDB single-digit-ms reads, and change data capture from every POS system.

2. **Near-real-time (< 5 minutes):** Stock updates batched every 1-5 minutes. Covers 95% of use cases at 60% of the cost. Uses SQS batching + Lambda.

3. **Periodic sync (15-60 minutes):** Scheduled ETL from store systems. Simplest but risks overselling for high-velocity SKUs.

My recommendation: **Near-real-time as default**, with true real-time for the top 50 high-velocity SKUs using EventBridge + DynamoDB Streams. This gives you the best cost-performance trade-off.

Before we design, I need to understand:

- How do stores currently report inventory? (POS API? File drop? ERP batch?)
- What's the source of truth — store POS or central ERP?
- Do you need cross-store visibility (e.g., ship-from-store)?
- What's the acceptable lag before it impacts customer experience?"

End of CGI L2 Technical Round Preparation

FILES IN THIS SERIES

File	Content
`AWS_Integration_Architect_Interview_Part1.md`	L1 Round: AWS services, integration patterns, security (Q1-Q24)
`AWS_Integration_Architect_Interview_Part2.md`	L1 Round: IaC, DevOps, retail scenarios (Q25+)
`CGI_L2_Technical_Round_Part1.md`	**L2 Round: EKS IAM (IRSA/Pod Identity), advanced AWS architecture, Saga pattern**
`CGI_L2_Technical_Round_Part2.md`	**L2 Round: Kubernetes deep-dive, validated K8s answers, scenario questions**

Prepared for: CGI AWS Integration Architect — L2 Technical Interview Candidate: Pushparaj Naik